Machine Learning Practical 2

PCA and LDA – Dimensionality Reduction | Python Code Only

```
# Practical 2: Dimensionality Reduction using PCA and LDA
# Dataset: Built-in Digits dataset from scikit-learn

# Install libraries if required:
# !pip install numpy pandas matplotlib scikit-learn

import time
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# 1. Load the dataset
digits = load_digits()
X = digits.data
y = digits.target

print("Dataset shape:", X.shape)
print("Number of classes:", len(np.unique(y)))

# 2. Split the dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Scale the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# 4. Apply PCA
# Reduce 64 original features to 30 features
pca = PCA(n_components=30)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print("\nPCA - Original features:", X_train_scaled.shape[1])
print("PCA - Reduced features:", X_train_pca.shape[1])
print("PCA - Explained variance:", round(
    np.sum(pca.explained_variance_ratio_), 4
))

# 5. Apply LDA
# LDA can have at most (number of classes - 1) components.
lda = LinearDiscriminantAnalysis(n_components=9)
X_train_lda = lda.fit_transform(X_train_scaled, y_train)
X_test_lda = lda.transform(X_test_scaled)

print("\nLDA - Original features:", X_train_scaled.shape[1])
print("LDA - Reduced features:", X_train_lda.shape[1])

# 6. Train and compare Logistic Regression models
results = []

datasets = {
```

```
    "Original": (X_train_scaled, X_test_scaled),
    "PCA": (X_train_pca, X_test_pca),
    "LDA": (X_train_lda, X_test_lda)
}

for name, (Xtr, Xte) in datasets.items():
    start_time = time.time()

    model = LogisticRegression(max_iter=2000)
    model.fit(Xtr, y_train)
    y_pred = model.predict(Xte)

    elapsed_time = time.time() - start_time
    accuracy = accuracy_score(y_test, y_pred)

    results.append({
        "Method": name,
        "Number of Features": Xtr.shape[1],
        "Accuracy": round(accuracy, 4),
        "Training Time (seconds)": round(elapsed_time, 4)
    })

# 7. Display comparison
results_df = pd.DataFrame(results)
print("\nComparison of Original, PCA and LDA:")
print(results_df)

# 8. Plot the comparison
plt.figure(figsize=(8, 5))
plt.bar(results_df["Method"], results_df["Accuracy"])
plt.title("Accuracy Comparison: Original vs PCA vs LDA")
plt.xlabel("Method")
plt.ylabel("Accuracy")
plt.ylim(0, 1.05)
plt.show()

# 9. Visualize PCA in two dimensions
pca_2d = PCA(n_components=2)
X_pca_2d = pca_2d.fit_transform(X_train_scaled)

plt.figure(figsize=(8, 5))
scatter = plt.scatter(
    X_pca_2d[:, 0], X_pca_2d[:, 1],
    c=y_train, cmap="tab10", s=15
)
plt.title("Digits Dataset after PCA (2 Components)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.colorbar(scatter, label="Digit")
plt.show()

# 10. Visualize LDA in two dimensions
lda_2d = LinearDiscriminantAnalysis(n_components=2)
X_lda_2d = lda_2d.fit_transform(X_train_scaled, y_train)

plt.figure(figsize=(8, 5))
scatter = plt.scatter(
    X_lda_2d[:, 0], X_lda_2d[:, 1],
    c=y_train, cmap="tab10", s=15
)
plt.title("Digits Dataset after LDA (2 Components)")
plt.xlabel("Linear Discriminant 1")
```

```
plt.ylabel("Linear Discriminant 2")
plt.colorbar(scatter, label="Digit")
plt.show()
```